

Informe Tecnico: Simulador de Atención de Soporte TechClassUC

Abelardo Ochoa

Luisa Mercado



Curso: Programación II

Ingeniería de Sistemas

Semestre: 2025-2

Docente: Alberto Paternina

Montería – Córdoba

2025

1. Arquitectura General

El sistema está basado en POO. La clase principal CustomerService administra:

- ArrayDeque<Client> – Cola de espera
- List<Client> – Lista de atendidos
- Stack<Action> – Registro de acciones

Controller coordina la interacción entre la GUI y el modelo.

2. Estructuras de Datos Utilizadas

2.1. Cola (ArrayDeque) – Gestión de Espera

Contiene a los clientes esperando atención.

Operaciones:

- addClient – agrega al final
- attendClient – extrae al primero (FIFO)
- removeClient – elimina un elemento específico

2.2. Lista (LinkedList / ArrayList) – Historial de Atendidos

Al finalizar la atención, el cliente pasa a la lista.

Permite:

- Búsqueda por ID
- Búsqueda por categoría
- Recorrido cronológico

2.3. Pila (Stack) – Registro de Acciones

Cada acción se guarda con:

- Tipo de acción
- Cliente involucrado
- Fecha/hora

Acciones posibles: ADD, REMOVE, ATTEND, UNDO.

3. Flujo Interno del Sistema

3.1. Llegada → Cola

El cliente es registrado y añadido al final.

3.2. Atención → Lista

Al atender un cliente:

- 1. Se extrae de la cola.*
- 2. Se guarda inicio de atención.*
- 3. Al finalizar, se mueve a la lista y registra el tiempo.*

3.3. Eliminación de Cliente

Si un cliente es eliminado desde la cola:

- Se extrae
- Se registra acción REMOVE para posible deshacer

4. Registro de Acciones

Cada operación importante del usuario genera un objeto Action, que queda registrado tanto para control como para auditoría.

La interfaz gráfica consulta esta información para mostrarla en el Historial de Acciones.

Las acciones se almacenan como objetos Action con:

- Tipo
- Cliente
- Fecha/hora

5. Función UNDO (Deshacer)

Cada acción se revierte de la siguiente forma:

- Si se deshace ADD:

Se elimina el último cliente agregado a la cola.

```
model.getWaitingClients().removeLast();
```

- Si se deshace REMOVE:

Se reinsertará el cliente en su posición original dentro de la cola.

Esto se hace usando:

-La pila removePositions, que guarda la posición real.

-El método insertInPosition(), que reconstruye la cola manteniendo el orden correcto.

```
model.setWaitingClients(insertInPosition(action.getClient(), model.getRemovePositions().pop()));
```

- Si se deshace ATTEND:

El sistema revierte completamente la atención:

1. El cliente es eliminado de la lista de atendidos.
2. Se devuelve al inicio de la cola.
3. Se elimina el tiempo registrado.
4. Se restaura el texto anterior mostrado al usuario.

```
model.getAttendedClients().removeLast();
```

```
model.getWaitingClients().addFirst(action.getClient());
```

```
model.getAttendTime().removeLast();
```

6. Módulo de Simulación

La simulación está implementada usando un Timer de Swing. Este funciona cuando el usuario presiona el botón “Simulación”, se ejecuta:

```
simulator();
```

El método:

1. Inicia un Timer con intervalos de 2500 ms.
2. Llama secuencialmente a diferentes pasos del sistema.
3. Cada paso ejecuta acciones reales como:
 - Agregar clientes.
 - Atender.
 - Finalizar atención.
 - Eliminar.
 - Deshacer.
 - Buscar.
 - Mostrar historial.

Es decir: simula un día completo automáticamente.

Cuando llega al último paso, el simulador se detiene solo.

7. Integración con la Interfaz Gráfica

El Controller actualiza:

- Tablas (waitingTable, attendTable, historyTable)
- Estadísticas (count, timeAverage)

Se usa un diseño desacoplado:

- Vista – Interacción
- Controlador – Lógica
- Modelo – Estructuras